

Hilfsmittel für das Modul 122

Befehle

Befehl	Wozu
cd	Wechselt das aktuelle Verzeichnis
cp	Kopiert eine Datei
crontab	Verwaltet die crontab-Aufträge (vom System ausgeführte Jobs)
echo	Gibt einen Text oder eine Variable aus
exit	Beendet die Ausführung eines Scripts / Schliesst die Shell
grep	Findet Suchbegriffe
ls	Zeigt alle Dateien in einem Verzeichnis
man	Zeigt die Manualseite für einen Befehl
mkdir	Erstellt ein Verzeichnis
mv	Verschiebt eine Datei oder benennt eine Datei um
pwd	Zeigt das aktuelle Verzeichnis an
read	Eingaben vom Terminal oder einer Datei lesen
rm	Löscht eine Datei oder ein Verzeichnis
rmdir	Löscht ein leeres Verzeichnis
sed	Bearbeitet Textdateien

Hinweise zu einzelnen Befehlen

sed

Wir haben nur die Befehle zum Löschen von Zeilen und zum Suchen/Ersetzen angeschaut. Dabei haben die folgenden Buchstaben als Befehle eine Bedeutung:

d Zeile löschen
s suchen & ersetzen
g jedes Vorkommen des gesuchten Ausdrucks ersetzen

crontab

Die einzelnen Positionen für einen Eintrag in der Cron-Tabelle (crontab):

```
* * * * * auszuführender Befehl

T T T T T
| | | | |
| | | |   Wochentag (0-7, Sonntag ist 0 oder 7)
| | |     Monat (1-12)
| |       Tag (1-31)
|         Stunde (0-23)
         Minute (0-59)
```

Beachten Sie, dass jeweils der absolute Pfadname des Befehls angegeben sein muss.

Kontrollstrukturen

if – einfache Verzweigungen

```
if test bedingung
then
    Befehl
else
    Befehl
fi
```

```
if [ bedingung ]
then
    Befehl
else
    Befehl
fi
```

Die else-Klausel ist optional. Das Wort „Befehl“ steht für eine beliebige Anzahl Befehle.

Die gebräuchlichsten Operatoren für die Formulierung der Bedingung finden Sie in der Tabelle unten.

Operator	Englisch	Bedeutung
<i>Vergleich von Zeichenketten (strings)</i>		
=		Gleich
!=		Ungleich
-z	zero	Leere Zeichenkette
-n	non-zero	Zeichenkette nicht leer
<i>Vergleich von numerischen Werten</i>		
-eq	equal	Gleich
-ne	not equal	Ungleich
-lt	less than	Kleiner als
-le	less or equal	Kleiner oder gleich
-gt	greater than	Grösser
-ge	greater or equal	Grösser oder gleich
<i>Abfrage von Dateiattributen</i>		
-e	exist	Existiert die Datei?
-r	read	Leserecht gesetzt
-w	write	Schreibrecht gesetzt
-x	execute	Ausführungsrecht gesetzt
-f	file	Normale Datei
-d	directory	Verzeichnis
!	not	Nicht

for – Schleifen

```
for variable in <liste>
do
    Befehl
done
```

<liste> kann eine mit Leerzeichen getrennte Liste von Werten oder von Dateinamen sein (Pfad und Platzhalter sind erlaubt).

Zwischen do und done können beliebig viele Befehle ausgeführt werden.

case – Mehrfachverzweigung

```
case variable in
    wert1) Befehl;;
    wert2) Befehl
        Befehl;;
    wert3 | wert4) Befehl;;
    *) Befehl;;
esac
```

Mit dem Zeichen | können mehrere Werte angegeben werden. Das Zeichen bedeutet „oder“.

Die Anzahl Befehle pro Fall ist nicht begrenzt. Eine Befehlsfolge muss mit ;; beendet werden. Dies entspricht dem break in Java.

*) steht für jeden beliebigen Wert.

while – Schleifen

<pre>while [bedingung] do Befehl done</pre>	<pre>while read variable do Befehl done < filename</pre>
---	---

Zwischen do und done können beliebig viele Befehle ausgeführt werden.

Befehlskette

Befehl | Befehl

Die Ausgabe des ersten Befehls wird zur Eingabe des zweiten Befehls. Es können beliebig viele Befehle auf diese Weise aneinander gehängt werden.

Umleitung der Ein- und Ausgabe

Umleitung	Bewirkt was
< filename	Es wird nicht von der Tastatur gelesen sondern aus einer Datei
> filename	Die Ausgabe wird in eine neue Datei geschrieben. (Existiert die Datei bereits, wird sie überschrieben.)
>> filename	Die Ausgabe wird einer Datei hinzugefügt.
2> filename	Fehlermeldungen werden in eine neue Datei geschrieben. (Existiert die Datei bereits, wird sie überschrieben.)
2>&1	Fehlermeldungen erscheinen in der gleichen Datei wie die normale Ausgabe

Reguläre Ausdrücke

Symbol	Bedeutung
[]	Zeichenklasse – enthält alle Zeichen, die an der aktuellen Position erlaubt sind (können auch als Bereich angegeben werden) – die Zeichenklasse steht nur für ein Vorkommen des Zeichens – Wiederholungen müssten mit entsprechenden Symbolen formuliert werden
\(\)	fasst Elemente eines Regulären Ausdrucks zu einer Gruppe zusammen (der Backslash vor der Klammer ist notwendig)
\b	(boundary) steht für eine Wortgrenze
\s	(space) steht für Whitespace (also Leerzeichen etc)
.	steht für jedes beliebige Zeichen
\.	steht für einen Punkt
\	trennt Alternativen (entweder das davor oder das dahinter) → am besten zu verwenden in einer Gruppe (der Backslash vor dem senkrechten Strich ist notwendig)
^	Zeilenanfang
\$	Zeilenende
\?	das Element davor kommt 0 oder 1 mal vor (der Backslash vor dem Fragezeichen ist notwendig)
*	das Element davor kommt 0 bis unendlich mal vor
\+	das Element davor kommt 1 bis unendlich mal vor (der Backslash vor dem Pluszeichen ist notwendig)
\{min, max\}	das Element davor kommt min bis max mal vor (max kann weggelassen werden, das Komma davor aber nicht) (der Backslash vor der Klammer ist notwendig)
\n	sed ersetzt dieses Symbol durch die n. Gruppe im Suchausdruck (n = 1, 2, 3, ...)